CS496 Software Project: Population Model Calculator

Justin Dorsey and Oselunosen Ehi-Douglas

2026-04-01

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Dr. Suzanne Keilson
- Client title: Associate Professor
- Client email address: skeilson@loyola.edu
- Client employer: Loyola University Maryland
- How you know the client: Student under her Probability and Statistics (EG-381) course

2 Project Description

2.1 Overview

The purpose of my project is to create a web-based tool that performs key calculations related to population models. The website will enable users to calculate values for exponential growth, carrying capacity, and differential equation–based models. In addition to this, the website will support interactive graphs, simulation playback, model comparison, and data fitting, giving students and instructors a more powerful and flexible tool than a simple calculator. The Project will have database features, this is so that users can create accounts, save models with parameters, view history of calculations, Re-run or re-graph previous models. The project will also have an instructor mode and student mode. The instructor will be able to create class example models, share model links with students, and see embedded graphs, while students will be able to run and save models. My client requested this project to support college students and faculty who work with these mathematical models in their coursework or research. Although other population-modeling tools exist, many of them can be difficult to access, difficult to use, or require complex software installations. This project aims to solve these problems by providing a convenient, easy-to-use, and interactive platform. The website simplifies input, minimizes user error, and displays results in a clear and readable format. Upload/download data sets. By focusing on accessibility and user-friendly design, the tool helps students and instructors perform population model calculations quickly and efficiently, reducing the learning curve associated with more advanced computational tools.

2.2 Key Features

- **Simple and Structured Data Input:** Easy and Organized Data Input User-friendly forms with error prevention, tool tips, and validation to reduce user errors.
- **Automatic Graph Generation:** Excellent charts with the ability to zoom, pan, and switch between linear and logarithmic axes were produced utilizing contemporary visualization libraries.

- Interactive controls Sliders and adjustable parameters that update graphs instantly, along with animation playback for time-dependent models.
- Multiple Model Options and Comparison Users can compare various parameter values, overlay graphs, and create several models side by side. Includes exponential and logistic growth, differential equations, carrying capacity, Lotka–Volterra predator–prey, and so on.
- Instructor Mode Professors can create example models and share them with students through special links or an instructor dashboard.
- User Accounts Users can create accounts to save their models, load previous calculations, and store graph images or parameters.
- Dataset Upload and Curve Fitting Users can upload CSV data (e.g., real population numbers) and the system will fit exponential or logistic curves automatically and provide error metrics.

2.3 Why this Project is Interesting

Initially, my client gave me three projects: A population model, a 4-year planner for engineering and computer science students, and A software tool that can take in MP3 or other audio formats, extract the audio envelope, transcribe the spoken content, and demodulate the signal. I chose the population model because, given that my client did not have all the details for the audio project, the population model seemed like the next best option in terms of low difficulty and the opportunity to apply my mathematical knowledge. I think this project might interest students in STEM. This is because there is a high possibility that in our academic journey, we could come in contact with topics that require population model calculations, and the project I chose could make it easier to solve those problems.

2.4 Areas of CS required

1. Web Development
2. CyberSecurity
3. Human Computer Interaction
4. Database Management
5. Software Engineering
6. Algorithm Analysis

2.5 Potential Concerns and Questions

I guess the things I am worried about in this project would be if I could complete the project in time and if the project would at the end meet the client's satisfaction. Feedback that I would love from my instructor would be the best programming languages to use for projects like this, and tips on how to break down the workload that comes with projects like this.

2.6 Summary of Efforts to Find a Project

I at first contacted my client via email 10/30/2025 and then I met with her face-to-face that same day. This is when she proposed to me three projects that I could work on. After that I met with her on a zoom call on 11/14/2025 which is when we finalized the population model as the project she wanted. I at first considered working on the Audio project but it did not workout because she said that it was more in a way research-based and that she did not really have all the information for the project yet. I did not contact other clients mainly because I was positive that my current client would have something that I could work on for her. However, I would say that it that doing that is not really the best option because if there were the possibility that my client did not have any projects I would have been stranded and possibly would not have been able to meet the deadline for having a software engineering project.

2.7 Comparison to Draft

I at first contacted my client via email on 10/30/2025, and then I met with her face-to-face that same day. This is when she proposed three projects for me to work on. After that, I met with her on a Zoom call on 11/14/2025, when we finalized the population model as the project she wanted. I at first considered working on the Audio project, but it did not work out because she said that it was more in a way research-based and that she did not really have all the information for the project yet. I did not contact other clients mainly because I was positive that my current client would have something that I could work on for her. However, I would say that doing that is not really the best option because if there were the possibility that my client did not have any projects, I would have been stranded and possibly would not have been able to meet the deadline for having a software engineering project.

3 Requirements

3.1 Non-Functional Requirements

ID	NFR Title	Category	Description
NFR1	Website design	Visual / Design	This system uses a clean, consistent, and accessible visual design, including a clear color scheme, readable fonts, and well-structured layouts to improve usability and reduce user confusion (Maroon & White).
NFR2	Authentication / Access Control	Security	This system shall enforce secure authentication and role-based access control to ensure that only authorized users can access protected features and data.
NFR3	Account Security	Security	This system shall protect user accounts by using secure session management, automatic session expiration, and industry-standard security practices to prevent unauthorized access.
NFR4	Usability / Learnability	Usability	This system shall be intuitive and easy to learn, minimizing user error through clear feedback, consistent interactions, and contextual guidance.
NFR5	Data Reliability / Persistence	Reliability	This system shall reliably store and preserve user data, ensuring saved models and history remain available and protected against data loss.
NFR6	Mathematical calculation	Usability	This system aims to compute population models accurately, ensuring mathematically correct results.

Table 1: Non-Functional requirements

3.2 Functional Requirements

ID	Story Title	Points	Description
U1	Login-in	2	As a user, I want to be able to log in, so I can access my models
U2	Sign Up	2	As a user, I want to be able to create an account so that I can store my data
U3	Forget password	2	As a user, I want to be able to reset my password so that I can gain back access to my account
U4	Settings screen	2	As a user, I want to be able to change anything in my profile, so that I can either delete, switch, and change anything for my account

ID	Story Title	Points	Description
A5	Comparing models	5	As an admin, I want to be able to compare multiple population Models side-by-side, so that I can understand how different assumptions lead to different long-term results.
U6	Saving models	3	As a user, I want to be able to save my population models, so that I can come back to them later.
U7	Model History	3	As a user, I want to be able to view past models and calculations that I saved, so that I can revisit or rerun previous analyses.
I8	Example models	2	As an instructor, I want to be able to create predefined population models with my students, so that I can demonstrate concepts during lectures.
I9	Shareable models	2	As an instructor, I want to be able to generate shareable links for models, so that the students can interact with the same parameters outside of class.
U10	Graphs	3	As a user, I want to be able to view generated graphs, so that I can see a visual representation of my models.
U11	Graph Controls	2	As a user, I want to zoom, pan, and switch graph scales, so that I can better analyze population trends.
U12	Export Graphs as Images	2	As a user, I want to export graphs as images or PDFs, so that I can include them in reports or assignments.
U13	Uploading CSV data	3	As a user, I want to be able to upload real enrollment or population data, so that I can compare simulated results with real-world trends.
A14	Manage roles	2	As an admin, I want to be able to assign and modify user roles, so that access to system features is properly controlled.
I15	Assign Assignments	3	As an instructor, I want to be able to export model parameters and graphs, so that I can include them in homework or exams.
A16	Restoring models	3	As an admin, I want to be able to restore user data, so that users can retrieve their previous models.
U17	Choosing models	2	As a user, I want to be able to select different types of run models from the website so I can run the calculations I want.
U18	Running models	2	As a user, I want to be able to run models, so that I can generate results and visuals of my input.
U19	Input Validation & Error Feedback	2	As a user, I want the system to validate model inputs and display clear error messages, so that I can correct mistakes before running a model.
U20	Tooltips / Help Content	2	As a user, I want tool-tips & explanation page for model parameters, so that I understand what each value represents.
I21	Instructor Model control	2	As an instructor, I want to control whether students can modify shared models, so that assignments remain consistent.
U22	Deleting Models	2	As a user, I want to be able to delete models, so I can free up space or remove unwanted models.
U23	Session Timeout/Auto Logout	2	As a user, I want to be automatically logged out after a period of inactivity so that my account is kept safe
U24	LogOut	1	As a user, I want to be able to log out manually, so I can safely exit the system
U25	Error Metrics	2	As a user, I want to see error metrics (RMSE), so that I can evaluate model accuracy.
U26	Curve Fitting Results	5	As a user, I want the system to automatically fit exponential or logistic curves to uploaded data, so that I can compare models with real data.

ID	Story Title	Points	Description
U27	Simulation Playback Controls	5	As a user, I want to play, pause, and scrub through model simulations over time, so that I can observe population dynamics.

Table 2: Functional requirements as User Stories

4 System Design

4.1 Architecture

Layered Architecture:

This system uses a layered architecture to separate responsibilities between the frontend and backend clearly.

Layered Architecture:

- *Controller Layer (Node.js API):* This layer serves as the entry point for all client requests. It receives requests from the React frontend and passes them to the appropriate service functions after validation.
- *Service Layer (Business Logic – Node.js):* This layer contains the core functionality of the system, including population model calculations (exponential, logistic, etc.), simulation execution, curve fitting and error metrics. It ensures that business logic is separated from request handling and is reusable across the application.
- *Data Access Layer (ORM → PostgreSQL):* This layer manages communication with the database. It performs CRUD operations using an ORM and abstracts database queries from the rest of the system.
- *Database Layer (PostgreSQL):* This is the persistent storage layer where all application data is stored, including user accounts, saved models, parameters, and datasets.

MODULES:

- User Management (UI)
- Controller
- Service Layer
- Data Access Layer
- Database Layer

4.2 Diagrams

This link takes you to a pdf of our Class Diagram based off the structure of what users have and don't have using the user stories: Project Assignment 2 PDF

4.3 Technology

This section outlines the core technologies used in the system, including Node.js for the backend, React + Vite for the frontend, PostgreSQL (ORM) for data storage, and supporting tools like MathJS, PapaParse, Plotly.js, and Jest to handle calculations, data processing, visualization, and testing.

Topic	Name
Backend	NodeJs
Calculations	MathJs
CSV Upload	PapaParse
DataBase	PostgreSQL (ORM)
Frontend	React + Vite
Graphs	Plotty.js
Testing	Jest

Table 3: Technology/Frameworks

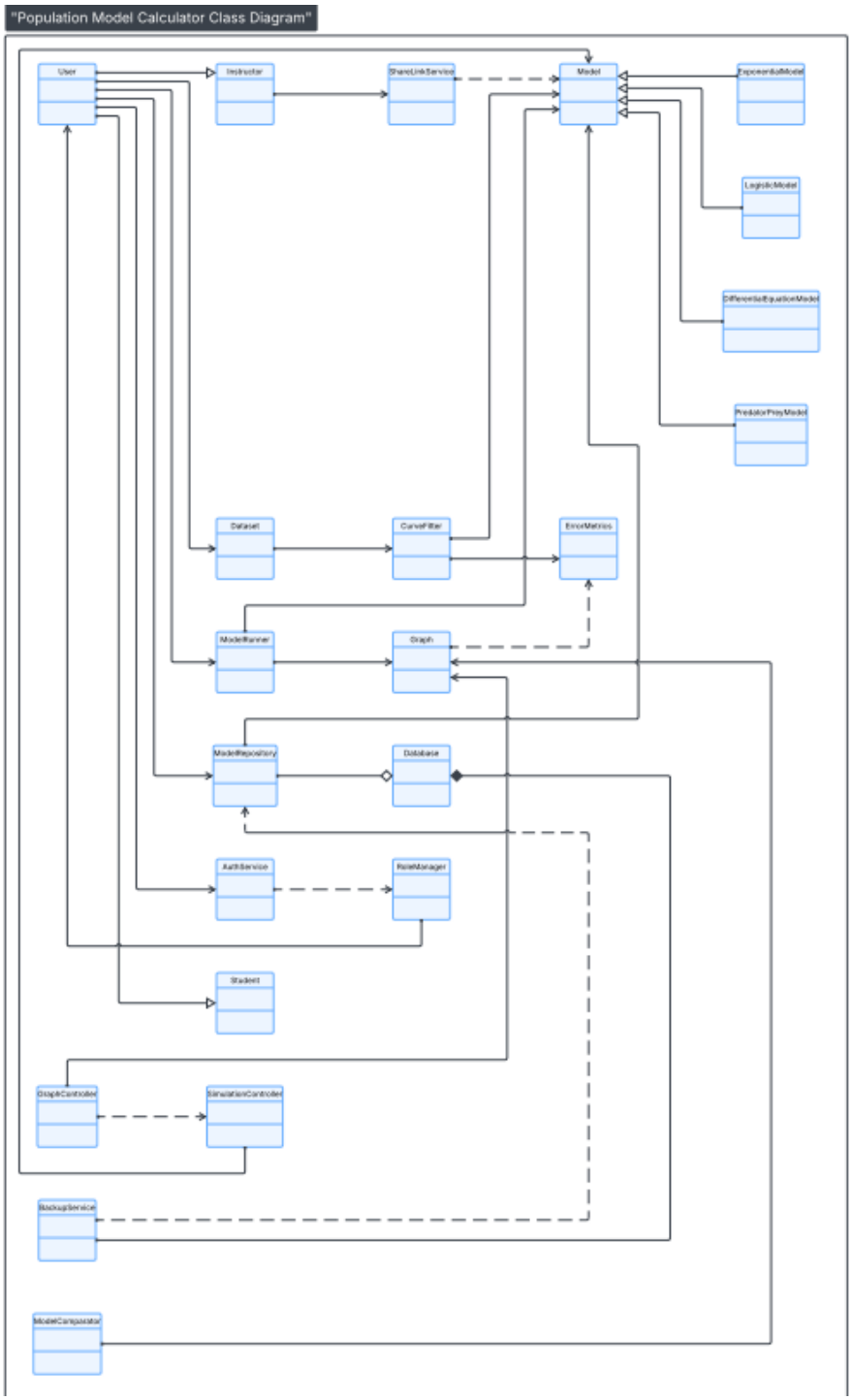


Figure 1: Class Diagram

4.4 Coding Standards

4.4.1 Naming

- **Global Variables:** All caps and underscores for spaces. Examples: `VARIABLE`, `TEST`, `MIN_NUM`
- **Classes:** Capitalized and CamelCase. Examples: `Class`, `ClassGuide`, `ExTwo`

- **Functions:** Lowercase with underscores for spaces. Avoid using numbers. Examples: `function`, `function_two`, `print_all_one`
- **Variables:** Same as functions, but numbers do not require underscores. Examples: `n`, `x`, `x2`, `print_status`
- **Files:** File names should be descriptive and follow variable naming rules unless another convention is necessary. Examples: `test_file.txt`, `grid_layout.py`, `App.jsx`
- **Folders:** Same as files. Keep names general and simple. Examples: `folder`, `folder_two`, `src`

4.4.2 Code Formatting

- Include a blank line between each function.
- Use consistent and readable loop structures.
- Place comments above functions and significant code blocks.
- Inline comments should be aligned to the right of the code line they describe.
- Maintain consistency throughout the codebase.

4.4.3 Commenting Rules

- Comments must be placed above each function explaining its purpose.
- Comments must be placed above each class.
- Comments should be easy to understand.
- Create and follow standard templates for class and function comments.

4.4.4 Coding Principles

- Follow the **DRY (Don't Repeat Yourself)** principle when designing code.

4.4.5 Testing

- Only allow code with unit tests and with at least 65% coverage to be committed.

4.5 Data

The database follows a relational structure where users create and manage population models. Each User can create multiple Population Models (one-to-many relationship). Each Population Model contains multiple Model Parameters, allowing different parameter values for different model types. Users can also upload Datasets, which may optionally be linked to a population model for curve fitting or comparison. Foreign keys are used to maintain these relationships and ensure referential integrity.

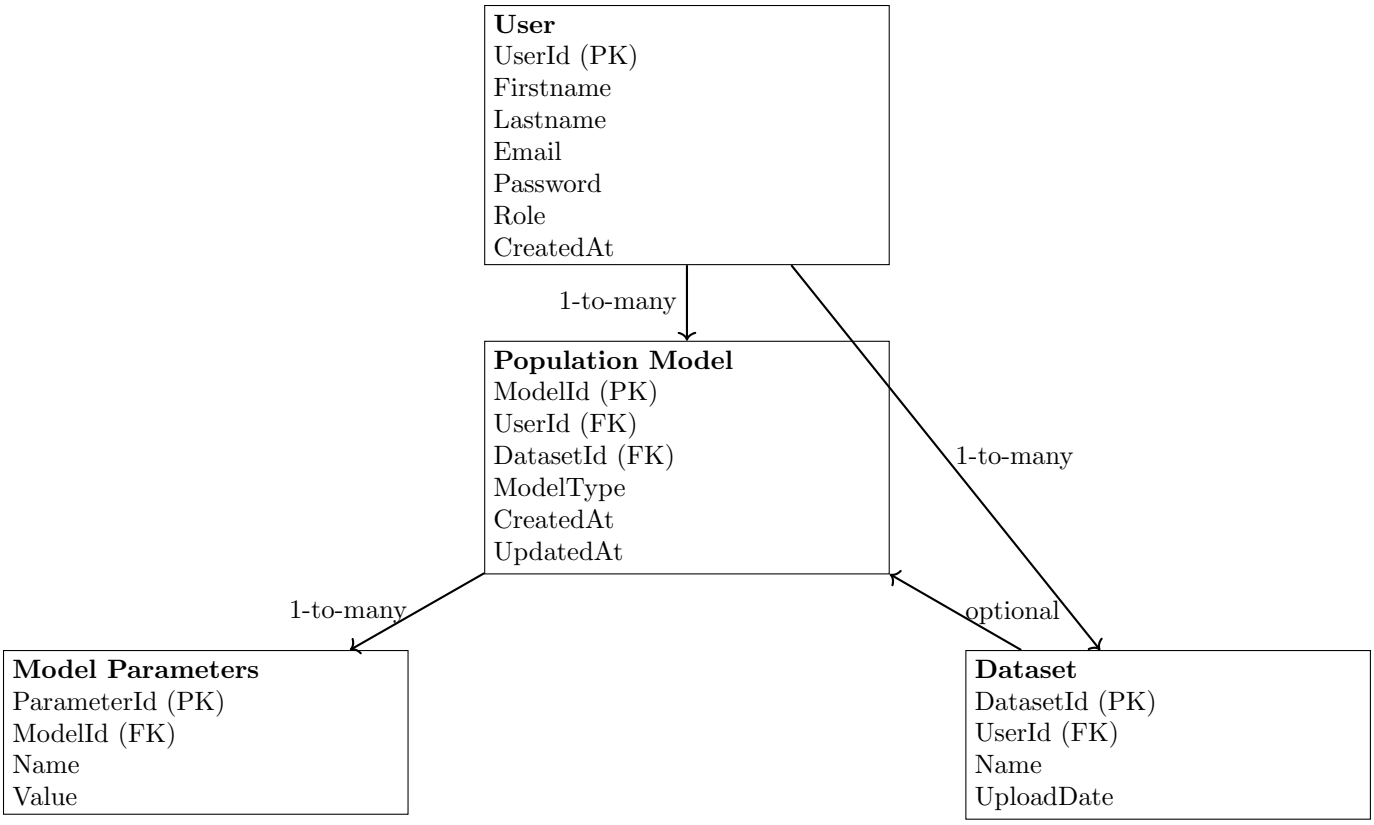


Figure 2: Entity-Relationship Diagram

4.6 UI Mocks

Figure 1, 2, and 3 are the ui examples, going from Figure 1 showing off the login, which in this example, ill login as a student. Figure 2 will show a working progress home page for a student, then in Figure 3 I picked the design model option which takes me to what figure 3 is.

5 Iterations

5.1 Iteration Planning

Table 4

Iteration	Dates	Stories	Points
1	02/5 - 02/12	U19, U20, U17, U25	8
2	02/13 - 02/24	U1, U2, U3, U10, U11	11
3	02/25 - 03/17	A5, U6, U7, A14	13
4	03/18 - 04/07	U12, U4, I8, I9, U27	13
5	04/9 - 04/22	I15, U13, U26, U18, I21, U22, U23, U24, A16	23
		Total:	68

Table 4: Iteration Planning for Incremental Deliveries

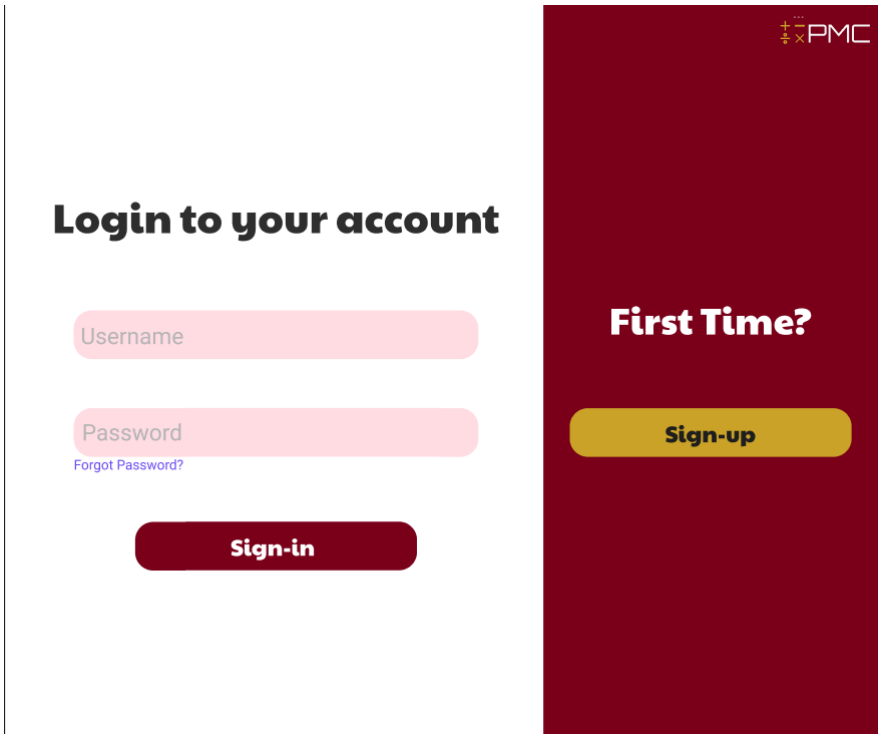


Figure 3: login screen with the ability to sign up and change passwords *(still working progress)*

5.2 Iteration/Sprint 1

5.2.1 Planning

Goal:

- ⋆ Justin: U20 Tooltips / Help Content 2pts
- ⋆ Ose: U25 Error Metrics 2pts
- ⋆ Ose: U17 Choosing models 2pts
- ⋆ Justin: U19 Input Validation / Error Feedback 2pts

5.2.2 Work Done

For this iteration, we could not complete any of the stories due to the snowstorm that derailed are school and meeting schedule. All we did for it was fully set up our frontend and backend. Justin cleaned up the frontend a bit, and Ose got started on his user story "A21," which is mainly backend development.

5.2.3 Testing Coverage

Figure 11: Testing was created for the functionality when the growth rate is missing and the functionality when the growth rate is not missing.

5.2.4 Retroespective & Reflection

What we learned from this is having a backup plan encase something like this happens again, which pushes back our progress from an iteration because all we got done was updating our GitHub with instructions on how to install the program and Kanban Board for our iteration 1. Justin has some base designs for our project, but not any for iteration 1, due to needing to buy the subscription for Figma, while Ose got everything ready to go when it came to the files and folders we need to begin the project, and even tried to get started on the backend mathematics required for JavaScript.

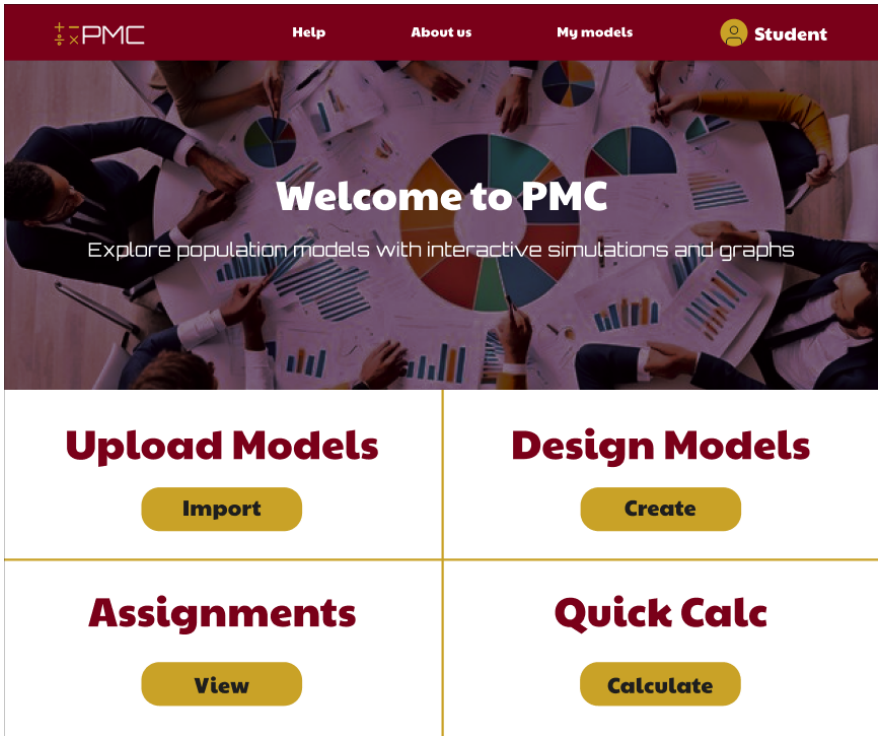


Figure 4: Homepage of the website for students *(working progress)*

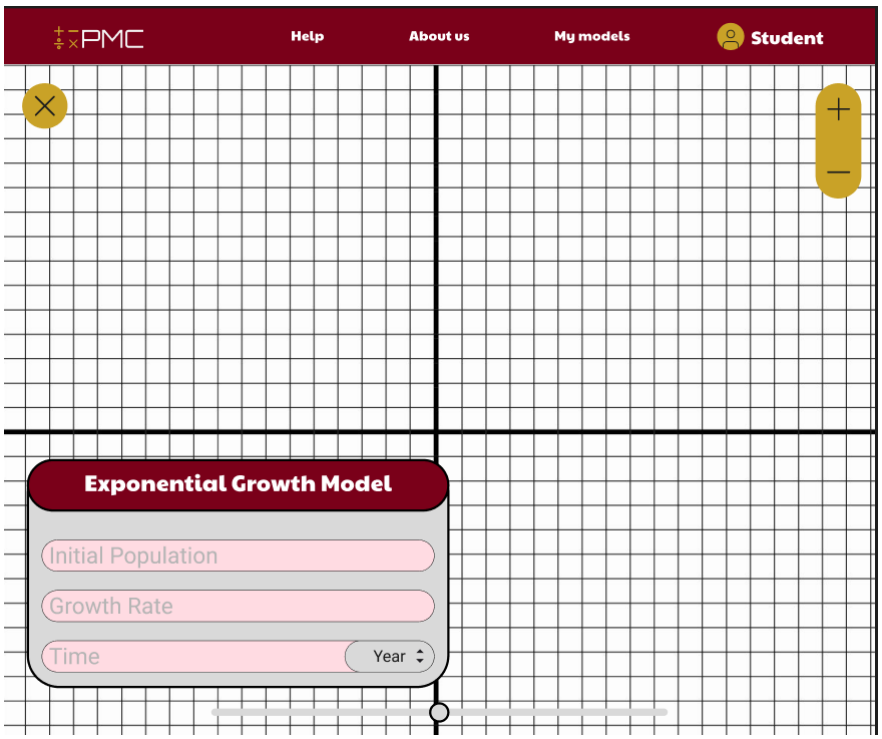


Figure 5: The creation page for your model with some of the features to it *(working progress)*

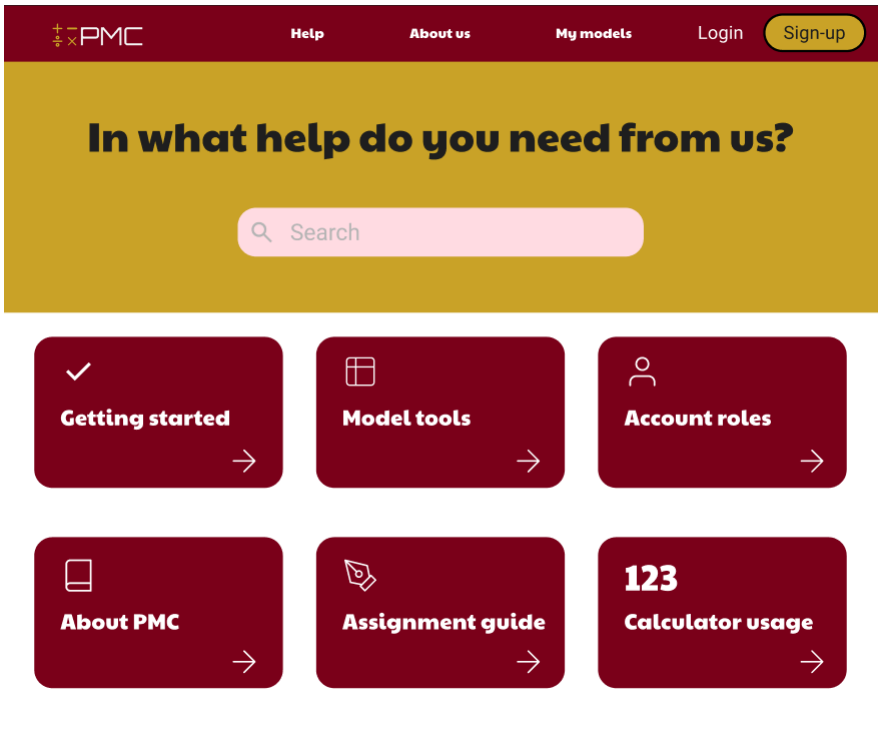


Figure 6: This help page is shown to have different information about using the website

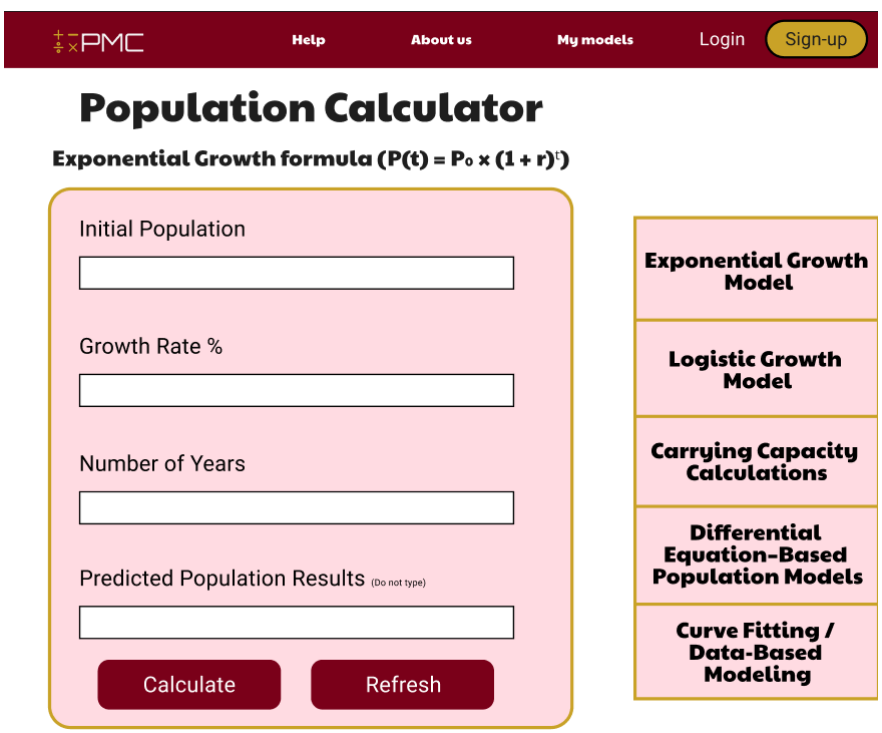


Figure 7: This is the only option for guest users to use when it comes to using our calculator

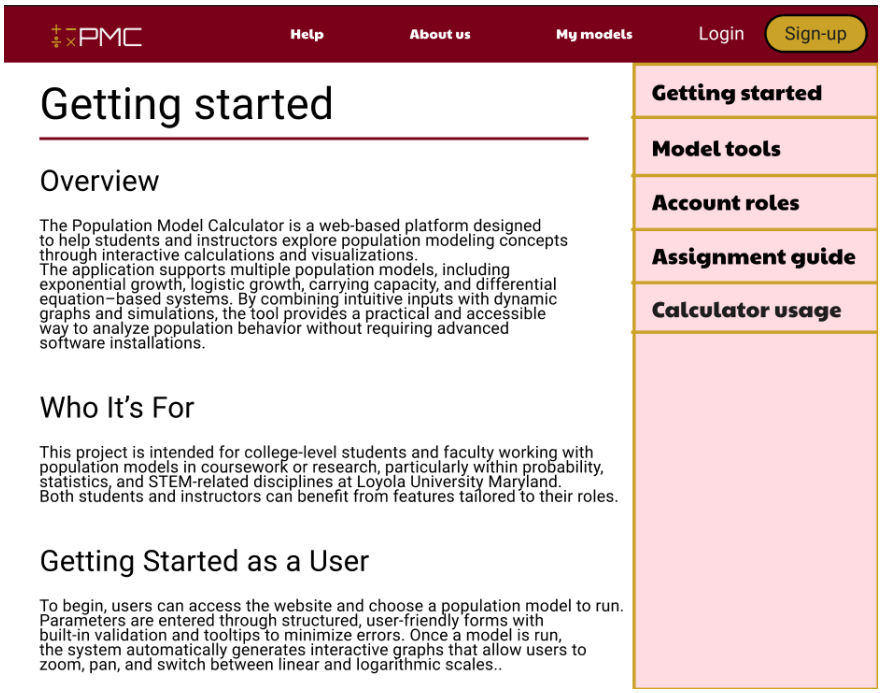


Figure 8: On the help page, this is one of the options that you can go to in it

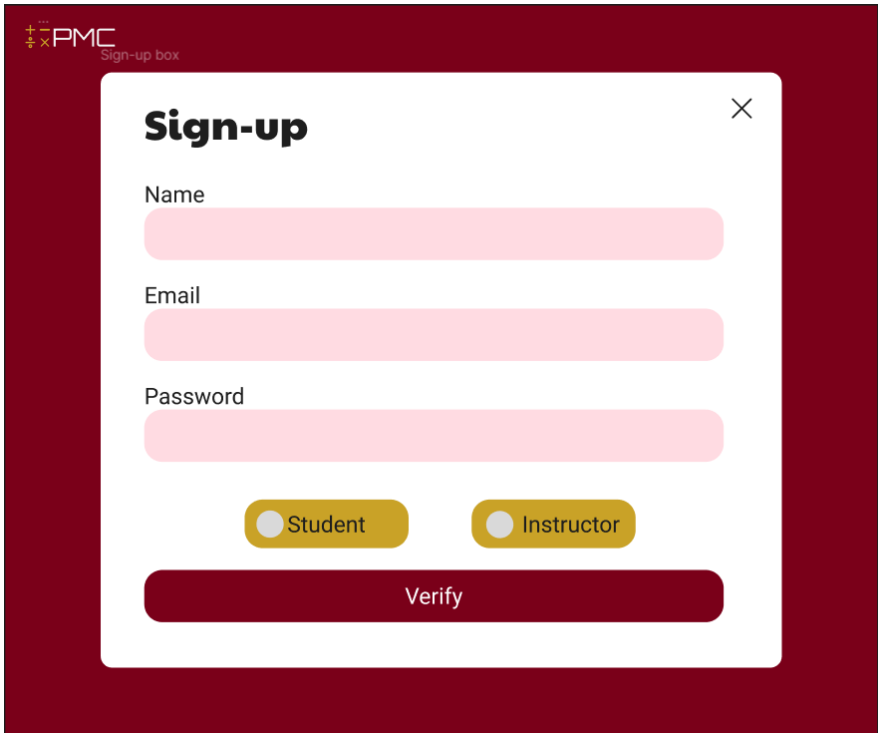


Figure 9: Once you click the sign-up, you'll be greeted to this page where you have to verify which user you'll be

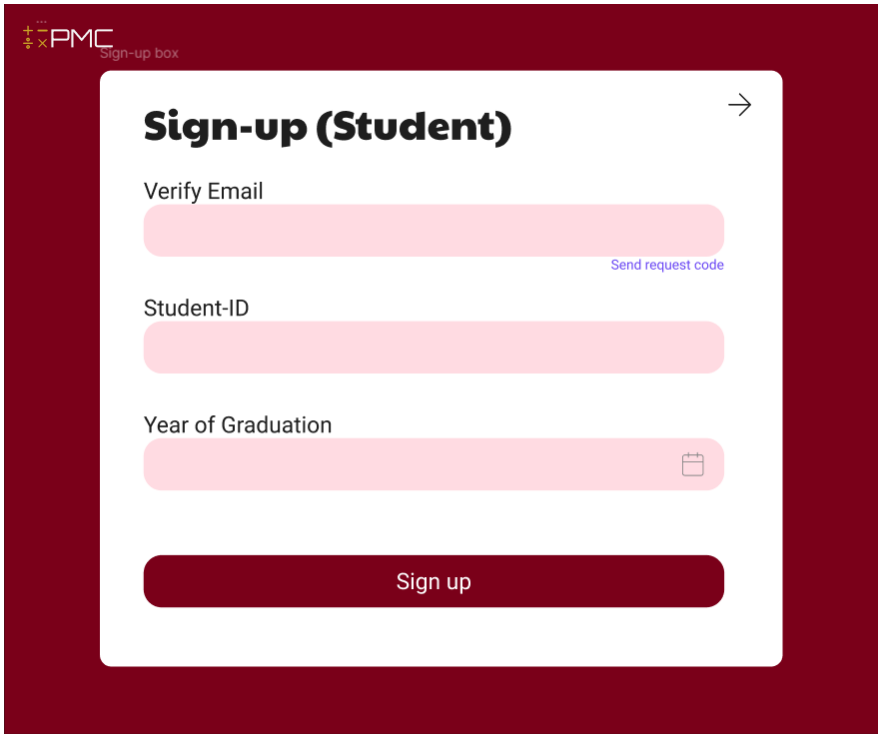


Figure 10: Once you verify under a student

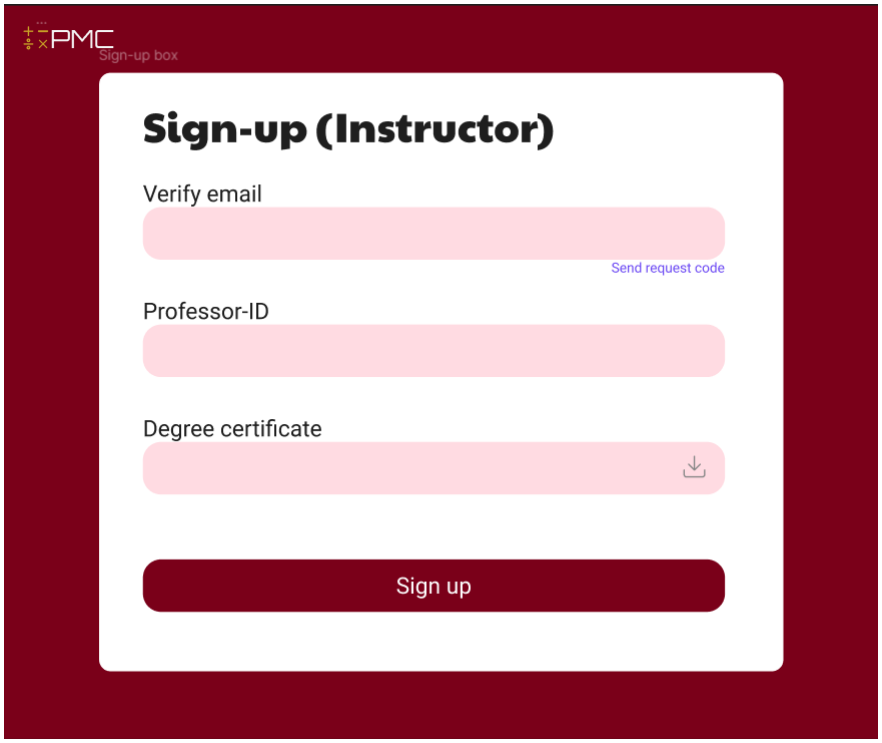


Figure 11: Once you verify under a instructor

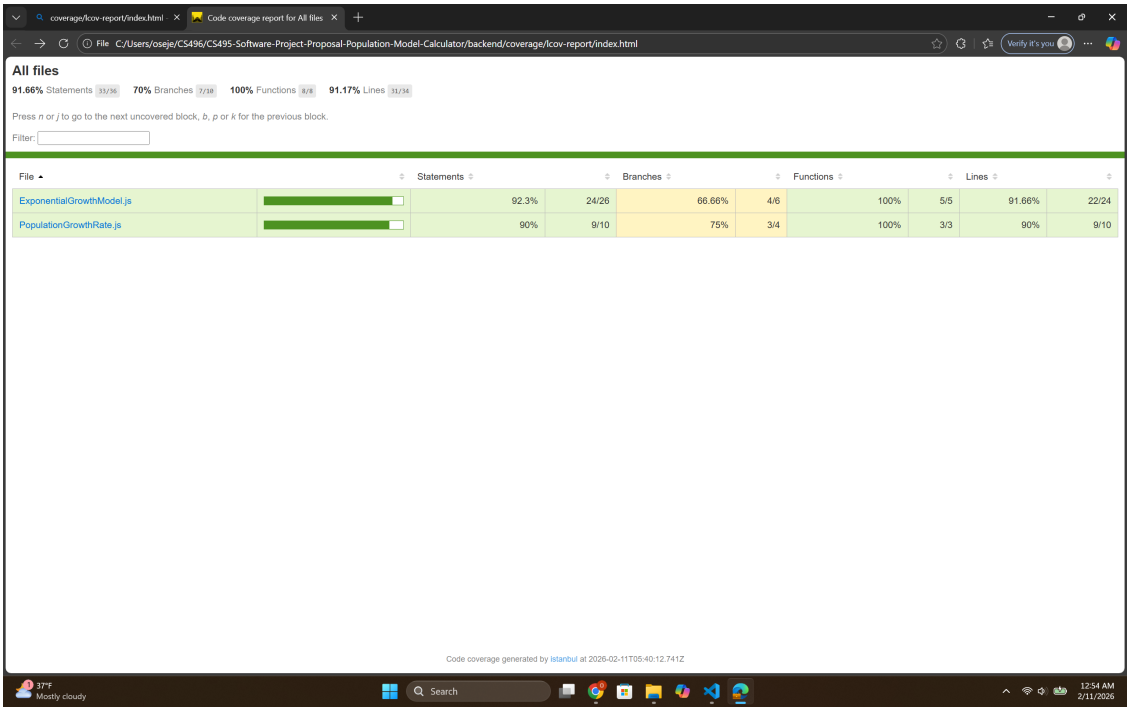


Figure 12: View of Backend test for the calculations we had done so far

5.3 Iteration/Sprint 2

5.3.1 Planning

Goal:

- ⋆ Ose: U10 Graphs 3pts
- ⋆ Ose: U11 Graph Controls 2pts
- ⋆ Justin: U3 Forget password 2pts
- ⋆ Justin: U1 Login-in 2pts
- ⋆ Justin: U2 Sign Up 2pts

5.3.2 Work Done

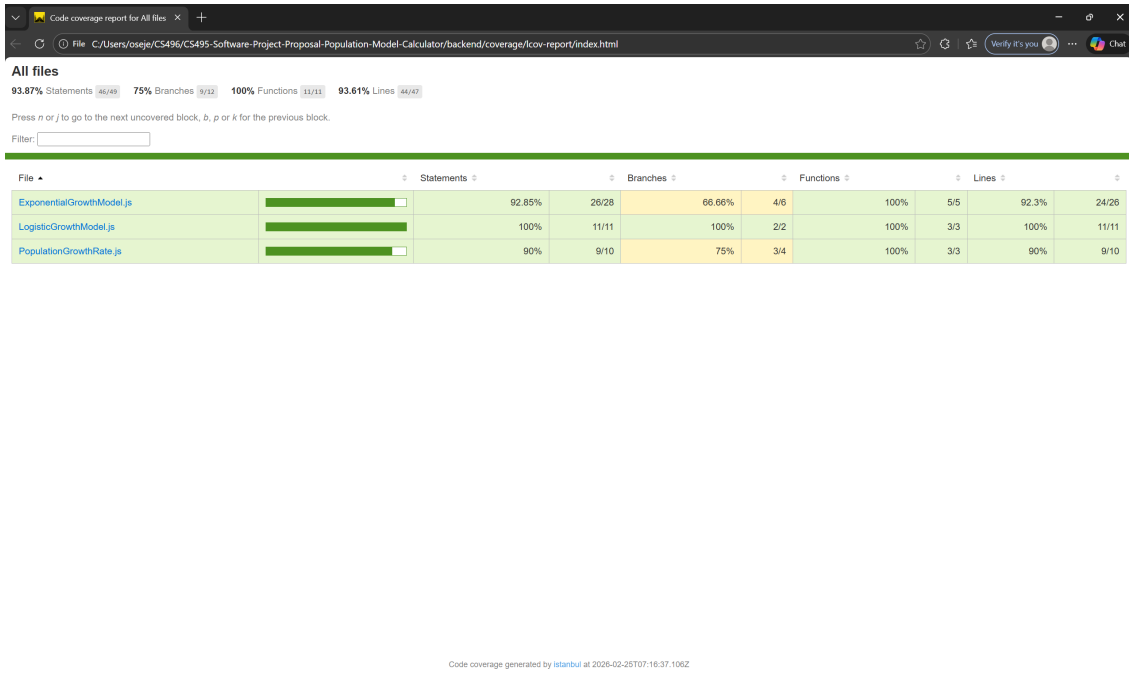
For this iteration, we got a good portion of the stories done and started this time around without having distractions to get in our way. What we did during this iteration was add more mock designs for our website (mainly being the different pov for student and professor users), while also setting new rules for going about letting each other know what's more important to get finished for every iteration. Justin started and finished both U1 and U2 while also starting U21 from the first iteration, and Ose added more to the NFR6, which was originally A21 while also starting to work on U10 on the backend.

5.3.3 Testing Coverage

Figure 12: The coverage report reflects how thoroughly the source code for LogisticGrowthModel, PopulationGrowth, and ExponentialGrowth has been executed by tests. Only the source files themselves are measured—test files are not counted.

5.3.4 Retroespective & Reflection

What we learned from this is knowing our time frame for doing certain user stories in our iteration, which means that we looked into which stories were easy enough to get done, while also being an important part of our website. Justin updated the document from the last assignment and made appropriate changes to the data, user stories, mocks, and



Code coverage report for All files

File C:/Users/oseje/CS496/CS495-Software-Project-Proposal-Population-Model-Calculator/backend/coverage/lcov-report/index.html

All files

93.87% Statements 46/49 **75%** Branches 9/12 **100%** Functions 11/11 **93.61%** Lines 44/47

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
ExponentialGrowthModel.js	92.85%	26/28	66.66%	4/6	100%	5/5	92.3%	24/26
LogisticGrowthModel.js	100%	11/11	100%	2/2	100%	3/3	100%	11/11
PopulationGrowthRate.js	90%	9/10	75%	3/4	100%	3/3	90%	9/10

Code coverage generated by istanbul at 2026-02-25T07:16:37.106Z

Figure 13: View of Backend test for the calculations we had done so far

NFRs. For programming-wise, Justin got a lot of the frontend things finished, but has things in the backend that need to match up with features that is shown in the frontend, while Ose was mainly getting his hands on setting up the backend of graphing which is the most important part of what our website done (he also added a few more things to the NFR6). Overall, our progress in this project gone a lot better in this second iteration, so that when we present, we'll have a live demo worth looking at.

5.4 Iteration/Sprint 3

5.4.1 Planning

Goal:

- ⋆ Justin: A5 Comparing models 5pts
- ⋆ Justin: U7 Model History 3pts
- ⋆ Ose: U6 Saving models 3pts
- ⋆ Ose: A14 Manage roles 2pts

5.4.2 Work Done

For this iteration, we got to working on the graph and role-based system of the stories, having some be more in the backend than the frontend. What we did during this iteration was add a homescreen for the create graph page, where you can see the history of past models or create one based on the four options from the quick calc page, with one of the create graph options available to continue to work on other iterations that are based around the graph. The backend of things was the main push for this iteration, surrounding the graph, besides fixing the different roles as a user. Justin started on both A5 and U7 (mainly being the format and design of it in the frontend), but the backend of things was not started because we came across a problem of not having a proper graph program based on the functions we designed the graph to do. Ose unfortunately could not get started on any of his iterations, which were mostly backend stuff that crossed paths with my user stories backend (mainly the model history). So, the total number of points we did this iteration was: Justin (A5 1.5pts and U7 2pts) / Ose (U6 1pt, A15 0pt, and A17 0pt).

5.4.3 Testing Coverage

For this current iteration, We would say that the testing is not good enough. Mainly due to the coverage being low. The plan to increase it would be to run more test that covers more lines for the different services and controllers created. So far the testing did not change much form last iteration. We created more .js files for iteration 3 and ran test on those.

All files

74.35% Statements 87/117 **52.27%** Branches 23/44 **89.47%** Functions 17/19 **73.45%** Lines 83/113

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
backend	83.33%	5/6	100%	0/0	100%	1/1	83.33%	5/6
backend/src/controllers	83.33%	5/6	100%	0/0	100%	1/1	83.33%	5/6
backend/src/services	73.33%	77/105	52.27%	23/44	88.23%	15/17	72.27%	73/101

Figure 14: General view of Test Coverage

All files backend/src/services

73.33% Statements 77/105 **52.27%** Branches 23/44 **88.23%** Functions 15/17 **72.27%** Lines 73/101

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
DiscreteGrowthModel.js	54%	27/50	43.33%	13/30	66.66%	4/6	53.06%	26/49
ExponentialGrowthModel.js	92.85%	26/28	66.66%	4/6	100%	5/5	92.3%	24/26
LogisticGrowthModel.js	88.23%	15/17	75%	3/4	100%	3/3	87.5%	14/16
PopulationGrowthRate.js	90%	9/10	75%	3/4	100%	3/3	90%	9/10

Figure 15: List of files in the controller folder that have test

All files **backend/src/controllers**

83.33% Statements 5/6 **100%** Branches 0/0 **100%** Functions 1/1 **83.33%** Lines 5/6

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
userControllers.js	83.33%	5/6	100%	0/0	100%	1/1	83.33%	5/6

Figure 16: List of Files in the services folder that have test

All files **backend**

83.33% Statements 5/6 **100%** Branches 0/0 **100%** Functions 1/1 **83.33%** Lines 5/6

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
db.js	83.33%	5/6	100%	0/0	100%	1/1	83.33%	5/6

Figure 17: List of files in the utils folder that have test

5.4.4 Retroespective & Reflection

What we learned from this is having more of an idea of what the backend will contain based on our designs and story that we updated, compared to how it was in the beginning. Seeing how this iteration was based on the graph system, it was considered to be the second biggest thing that our project would have so not getting any good progress on the backend to it took a heavy blow to its progress. Justin updated the document from the last assignment and made appropriate changes to the data, architecture, diagram, and points on two of the user stories. For programming-wise, Justin got to work on what the graph page will look like and the contents of how to create or see your graphs, but any of the backend features like saving the graph, comparing two graphs at the same time, and having an actual graph program for it were not there yet. Ose updated one part of the document, which was the "Summary of Efforts to Find a Project". For programming-wise, Ose went back and got the backend of the sign-in story finished and added more files to test on (also got the database fully working now). Overall, our progress in this iteration has gone down for reasons that we'll have to meetup discuss about but once it's been address the progress of this project will go back up.

5.5 Iteration/Sprint 4

5.5.1 Planning

Goal:

- ⋆ Ose: U12 Export Graphs as Images 2pts
- ⋆ Justin: U4 Settings screen 2pts
- ⋆ Ose: I8 Example models 2pts
- ⋆ Ose: I9 Shareable models 2pts
- ⋆ Justin: U27 Simulation Playback Controls 5pts

5.5.2 Work Done

For this iteration, we got to work on the graph's features and the user's account of the stories, having some be more in the backend than the frontend. In this iteration, we added a settings screen to your profile that lets you change or update your account and delete it. We also continued work on the graph system, mainly on what you can do once you create a graph from our website, which includes backend functionality. Justin started on U4 but could not get started on U27 (Ose stepped in and did the layout for one of the calculators that had it). For U4, he managed to get everything finished for what that screen is going to do, and for U27 he could not get that started without having a graph saved to the website. Ose did not get started on anything from this iteration, but instead went back to finish loose ends from previous iterations. What he did was the backend of two of the calculations, the frontend of both the login/signup screen, JWT authentication for accounts, graph simulation (for the calculator), and testing from the continuous growth model. So, the total number of points we did this iteration was: Justin (U4 2pts and U27 3pts(Ose points)) / Ose (U12 0pt, I8 0pt, and I9 0pt).

5.5.3 Testing Coverage

For this current iteration, We would say that the testing is better than last iteration. Although we were not able to work on user stories for this iteration we were able to improve on last iteration to gain better coverage on the previous stories. Figure 18 gives a general overview of the test coverage showing 4 folders and within those folders are the files that have test files. The backend folder contains the db.js as seen in Figure 19. The controllers folder contain the test for our database files for creating users, storing models and login and logout as seen in figure 20. The services folder is where the files for our calculations are in Figure 21. Finally, the Utils folder contain helper files which were used together with the files in our services folder; Figure 21. Some test are low because we had not the time to write test for them yet but hopefully we plan to be able to.

All files

75.3% Statements 247/328 **61.37%** Branches 89/145 **84.78%** Functions 39/46 **76.02%** Lines 241/317

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
backend	85.71%	6/7	100%	0/0	100%	1/1	85.71%	6/7
backend/src/controllers	69.04%	58/84	52%	13/25	100%	9/9	69.04%	58/84
backend/src/services	98.11%	156/159	85.71%	48/56	92%	23/25	99.34%	151/152
backend/src/utils.js	34.61%	27/78	43.75%	28/64	54.54%	6/11	35.13%	26/74

Code coverage generated by istanbul at 2026-04-01T05:31:56.600Z

Figure 18: General view of Test Coverage

All files backend

85.71% Statements 6/7 **100%** Branches 0/0 **100%** Functions 1/1 **85.71%** Lines 6/7

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
db.js	85.71%	6/7	100%	0/0	100%	1/1	85.71%	6/7

Figure 19: List of files in the backend folder that have test

All files backend/src/controllers

69.04% Statements 58/84 **52%** Branches 13/25 **100%** Functions 9/9 **69.04%** Lines 58/84

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
LoginController.js	100%	16/16	100%	8/8	100%	2/2	100%	16/16
modelController.js	71.69%	38/53	50%	3/6	100%	6/6	71.69%	38/53
userControllers.js	26.66%	4/15	18.18%	2/11	100%	1/1	26.66%	4/15

Code coverage generated by istanbul at 2026-04-01T05:31:56.600Z

Figure 20: List of files in the controller folder that have test

All files **backend/src/services**

98.11% Statements 156/159 **85.71%** Branches 48/56 **92%** Functions 23/25 **99.34%** Lines 151/152

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
ContinuousGrowthModel.js	100%	30/30	90%	9/10	100%	5/5	100%	29/29
DiscreteGrowthModel.js	100%	53/53	80.76%	21/26	100%	7/7	100%	52/52
EMComparison.js	92.3%	24/26	100%	4/4	60%	3/5	100%	22/22
LogisticGrowthModel.js	100%	40/40	91.66%	11/12	100%	5/5	100%	39/39
PopulationGrowthRate.js	90%	9/10	75%	3/4	100%	3/3	90%	9/10

Code coverage generated by istanbul at 2026-04-01T05:31:56.600Z

Figure 21: List of Files in the services folder that have test

All files **backend/src/utils.js**

34.61% Statements 27/78 **43.75%** Branches 28/64 **54.54%** Functions 6/11 **35.13%** Lines 26/74

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
ErrorMetrics.js	0%	0/23	0%	0/2	0%	0/4	0%	0/21
GrowthRate.js	54.54%	12/22	50%	19/38	100%	3/3	54.54%	12/22
TimeChecker.js	45.45%	15/33	37.5%	9/24	75%	3/4	45.16%	14/31

Code coverage generated by istanbul at 2026-04-01T05:31:56.600Z

Figure 22: List of files in the utils folder that have test

5.5.4 Retroespective & Reflection

The design decisions we made during this iteration was finishing all the features that our graph would have and slowly start moving to the smaller things we need for the project, the main reason we took that as our approach is because we knew in advance that the whole graphing stage of things would need a lot time to work on so for that we moved graph related things towards to the middle/end of our iterations in a group. The difficult thing about that is when certain aspects of the graph are not finished, or even the skeleton of it is not done, it's pretty challenging to work on stories that are based on an example of a graph. Looking back at previous iterations, having certain stories done would have made this iteration a lot easier to work around, but even still, the layout of things should have been done rather than just avoiding it. Unfortunately, the points didn't really match with this current iteration besides two, unless you count going back to start old iteration stories and finishing them. You can say that we underestimate certain stories in this one. For something new we came across was updating the database for the backend, which can help in making accounts for us to use and test other things that need that verification, and using ploty.js more and learning that you can customize certain controls that will pair well with that graphing program. On to what we delivered, it can be seen as workable, but not its final stage of what it's going to look like. Communication was one-sided at times, but once the other person is reminded of what to do, communication is back at 100. That work this sprint can stem from getting one or two things done while also having a bunch of stuff from previous iterations done or started on. Now, what we would do differently is remind each other of what to do daily (even during break) because we can not have one person getting points while the other is doing something else.

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[]

5.6.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project.]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently. How can you do better work in future projects? You may write this as a team or per person (or both)]

1. Missing or Underrepresented User Stories

These are features clearly implied by your project description or NFRs but not explicitly covered in Table 2.

🔐 Authentication & Security (NFR2, NFR3)

Missing Story: Email Verification

> *As a user, I want to verify my email after signing up, so that my account is secure and recoverable.*

Why it's missing:

- You have login, signup, and password reset, but no verification step.
- This directly supports **Security** and **Permission Control**.

Suggested points: **2**

Missing Story: Session Timeout / Logout

> *As a user, I want to be automatically logged out after inactivity, so that my account remains secure.*

Why it's missing:

- Security is listed as an NFR, but no functional story enforces it.

Suggested points: **1–2**

Figure 23: A descriptive caption for the image.

📊 Graphing & Visualization (Key Feature: Interactive Graphs)

Missing Story: Export Graphs as Images

> *As a user, I want to export graphs as images or PDFs, so that I can include them in reports or assignments.*

Why it's missing:

- You mention "store graph images" in the description.
- I13 exports parameters, but not clearly graphs themselves.

Suggested points: **2–3**

Missing Story: Axis & Scale Controls

> *As a user, I want to switch between linear and logarithmic scales, so that I can better analyze growth behavior.*

Why it's missing:

- Explicitly listed in Key Features.
- Currently only implied inside "model controls".

Suggested points: **2**

Figure 24: A descriptive caption for the image.

🧠 Learning & Usability (NFR4)

Missing Story: Tooltips / Help Explanations

> *As a student, I want tooltips or explanations for model parameters, so that I understand what each val represents.*

Why it's missing:

- NFR4 ("Viewing / Learning") is very broad.
- This is critical for a **student-facing educational tool**.

Suggested points: **2–3**

Missing Story: Error Feedback for Invalid Input

> *As a user, I want clear error messages when inputs are invalid, so that I can correct mistakes easily.*

Why it's missing:

- You mention "error prevention" in features, but no explicit story.
- This also supports HCI and usability grading.

Suggested points: **2**

📁 Data & History (NFR5)

Missing Story: View Model History

> *As a user, I want to view a history of my past models, so that I can revisit or rerun previous analyses.*

Figure 25: A descriptive caption for the image.

📁 Data & History (NFR5)

Missing Story: View Model History

> *As a user, I want to view a history of my past models, so that I can revisit or rerun previous analyses.*

Why it's missing:

- You mention "view history of calculations" in the overview.
- Saving models ≠ browsing history.

Suggested points: **2–3**

Missing Story: Delete or Archive Models

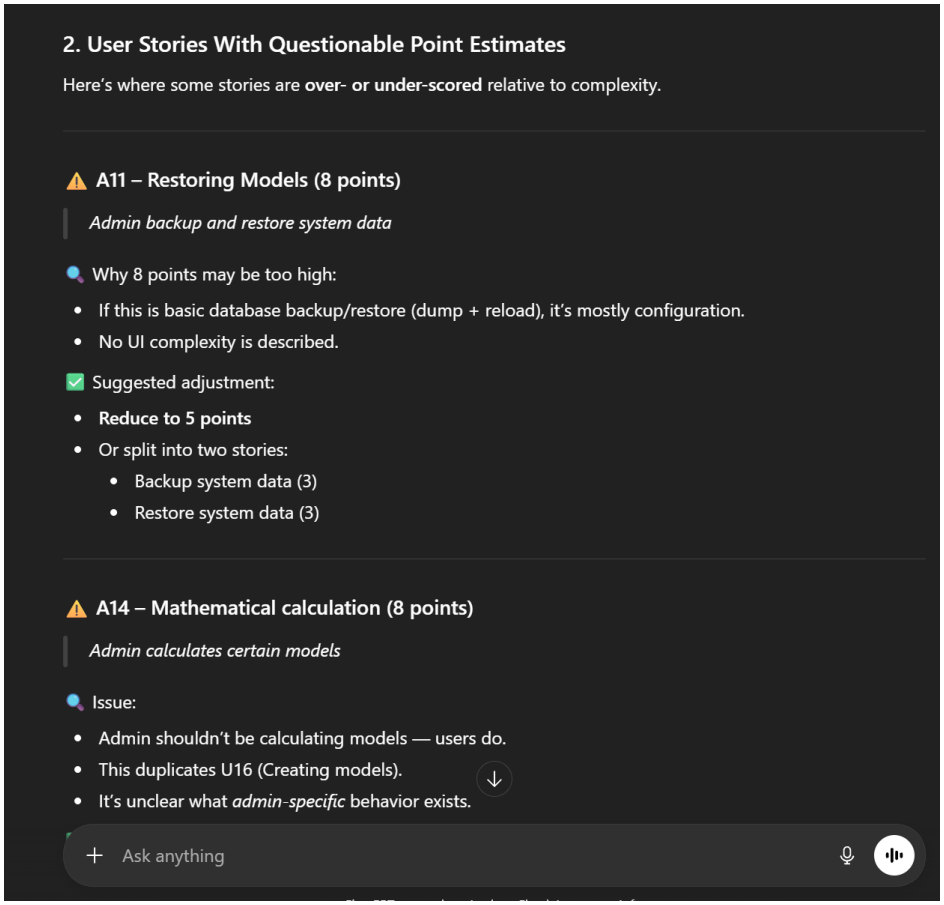
> *As a user, I want to delete or archive saved models, so that I can manage my workspace.*

Why it's missing:

- Maintainability implies lifecycle management.
- Prevents clutter and supports long-term use.

Suggested points: **1–2**

Figure 26: A descriptive caption for the image.



2. User Stories With Questionable Point Estimates

Here's where some stories are **over- or under-scored** relative to complexity.

⚠️ A11 – Restoring Models (8 points)

> *Admin backup and restore system data*

🔍 Why 8 points may be too high:

- If this is basic database backup/restore (dump + reload), it's mostly configuration.
- No UI complexity is described.

✅ Suggested adjustment:

- **Reduce to 5 points**
- Or split into two stories:
  - Backup system data (3)
  - Restore system data (3)

⚠️ A14 – Mathematical calculation (8 points)

> *Admin calculates certain models*

🔍 Issue:

- Admin shouldn't be calculating models — users do.
- This duplicates U16 (Creating models).
- It's unclear what *admin-specific* behavior exists.

Figure 27: A descriptive caption for the image.

🔍 Issue:

- Admin shouldn't be calculating models — users do.
- This duplicates U16 (Creating models).
- It's unclear what *admin-specific* behavior exists.

✅ Suggested fix:

- **Reframe** as:

  > *As a system, I want to accurately compute population models, so that results are mathematically correct.*

- Move this to a **technical story** or reduce to **3–5 points**.

⚠️ U16 – Creating models (8 points)

This one is actually **justified**, but...

🔍 Why it works:

- Multiple model types
- Parameter validation
- UI + backend logic

✅ Recommendation:

- Keep **8 points**
- But consider splitting later:
  - Create exponential/logistic models (5)
  - Create differential equation models (5)

Figure 28: A descriptive caption for the image.

3. Mapping Back to Non-Functional Requirements (for your write-up)

You can justify the missing stories like this in your report:

- **NFR2 (Permission Access)**
  → Email verification, session timeout, role-based access
- **NFR3 (Security)**
  → Password reset, logout, verification, admin controls
- **NFR4 (Viewing / Learning)**
  → Tooltips, help text, error messages, readable graphs
- **NFR5 (Maintainability)**
  → Model history, archiving, backups, deletion

This strengthens your document because reviewers love seeing **NFRs clearly enforced by functional stories**.

Quick Verdict (Instructor-Proof Summary)

- Your core stories are solid and aligned with the client's goals.
- You are **missing usability, learning, and lifecycle stories**, which are crucial for an educational tool.
- A few admin stories are **over-scoped or misassigned**.
- Adjusting points and adding 6–8 lightweight stories will make the backlog feel realistic and professionally engineered.

If you want, next we can:

- Rewrite the **missing stories in the exact table format**, or
- Help you **rebalance points to hit your iteration totals cleanly**, or
- Prep a **short justification paragraph** for your CS496 submission.

Figure 29: A descriptive caption for the image.

7 AI Usage

7.1 Reflection

I believe that the user stories given by the AI some where very helpful. ChatGpt helped me in the fact that some features that I did specify in my overview I did not add to my user stories and those where pointed out to me by ChatGpt when I used it. One of those examples would be U29 (Curve Fitting Results) and U30 (Simulation Playback Controls) which are important to the UI design of the project. It also helped in pointing out redundant stories and stories that had too many points which I in turn had to correct.

Appendix

[Appendix section if needed]